

Arquitetura e Planejamento: Detecção de Fraude em Transações Financeiras

1. Visão Geral do Problema e Objetivo

O Problema: Instituições financeiras enfrentam perdas milionárias anuais decorrentes de fraudes em cartão de crédito e transações digitais. Regras rígidas manuais geram altas taxas de Falsos Positivos (bloqueio indevido de clientes legítimos), gerando atrito no atendimento e insatisfação.

Objetivo da IA: Treinar um modelo de inteligência artificial supervisionado capaz de prever se uma transação financeira é fraudulenta (classe 1) ou legítima (classe 0) a partir de atributos comportamentais e contextuais em tempo real.

Métrica de Sucesso (Negócio): Maximizar a captura de fraudes (Recall > 50%) mantendo a precisão competitiva (F1-Score > 0.20), reduzindo o custo financeiro combinado de perdas por fraude e atrito de suporte.

2. Coleta e Preparação de Dados

Fontes de Dados: Gateway de Pagamentos, Logs de Sessão de Dispositivo, Cadastro transacional.

Atributos Principais: `valor_transacao`, `hora_do_dia`, `distancia_do_local_habitual_km`, `transacoes_ultimas_24h`, `dispositivo_novo`.

Engenharia de Features: Normalização MinMax, variáveis binárias de horário de madrugada e alta frequência de compras, e interações não-lineares triplas.

Tratamento de Desbalanceamento: O conjunto apresenta ~4% de fraudes. Aplicou-se **Oversampling** (reamostragem com reposição) no conjunto de treino para balancear a retropropagação da rede neural.

3. Estratégia de Bases e Separação de Dados

Armazenamento: Data Warehouse / Cloud Storage (BigQuery).

Separação de Dados: 70% Treino e 30% Teste estratificado via *Stratified Train-Test Split*.

Prevenção de Vazamento: Escalonador e Oversampling calculados exclusivamente sobre o conjunto de treino.

4. Seleção de Algoritmos

Modelo Baseline: Regressão Logística com `class_weight='balanced'` (Modelo linear interpretável).

Modelo Avançado: Rede Neural Multi-Layer Perceptron (MLP) com camadas ocultas (16, 8) e ativações ReLU.

Justificativa: A Rede Neural é capaz de capturar interações não-lineares complexas de alta ordem entre variáveis que a regressão linear isolada não aprende.

5. Estratégia de Treinamento e Otimização

Função de Custo: Log-Loss (Cross-Entropy) otimizada com algoritmo *Adam*.

Parâmetros: `hidden_layer_sizes=(16, 8)`, `activation='relu'`, `max_iter=500`, `early_stopping=True`.

6. Testes, Validação e Métricas Multi-Seed

Modelo	Precisão (Média ± DP)	Recall (Média ± DP)	F1-Score (Média ± DP)
Regressão Logística (Baseline)	0.096 ± 0.002	0.607 ± 0.013	0.166 ± 0.002
MLP (Sem Oversampling)	0.182 ± 0.315 (Colapsa)	0.016 ± 0.028 (Colapsa)	0.029 ± 0.051 (Colapsa)
MLP (Com Oversampling)	0.138 ± 0.019	0.514 ± 0.009	0.217 ± 0.024 (Vencedor)

Otimização de Custos de Negócio: Ajuste do limiar de probabilidade (`predict_proba`) baseado na matriz de custos: *Custo Total* = (*Falso Negativo* * R\$ 1.500) + (*Falso Positivo* * R\$ 100).

7. MLOps: Deploy e Monitoramento

Implantação: Microsserviço de inferência via API REST em container Docker (latência < 30ms).

Monitoramento: Acompanhamento contínuo de *Data Drift* nas variáveis de entrada e recalibração semanal contra *Concept Drift*.